

Calculator Project: a beginner-friendly guided tour

A guided explanation of the React + Django calculator

1. What you built

This is a small full-stack web application: a React user interface sends a math expression to a Django REST API, and Django returns a result. For example, when the user enters $2+3 \times 4$ (the screen uses the multiplication symbol $\times$), the browser sends JSON to the API. The API validates and evaluates it, then replies with JSON containing 14.

The application has three layers:

- Browser (React + TypeScript + CSS, port 5173)
 - > HTTP POST with JSON
- Django REST API (Python, port 8000)
 - > SQL when history is used
- PostgreSQL database

The calculator works without writing history to the database today. The Calculation model and migration are the prepared foundation for a future history feature.

2. Concepts before files

Frontend, backend, and API

The frontend is the part people see and click. It runs in the browser. React helps it redraw the screen when data changes. TypeScript is JavaScript with extra checks while you develop.

The backend runs on a server, not in the browser. Django receives requests, applies business rules, and replies. An API is the agreed interface between the two programs. Here the agreement is:

```
POST /api/v1/calculate/  
Content-Type: application/json
```

```
{ "expression": "2+3x4" }
```

```
Success: { "result": "14" }
```

```
Invalid input: { "detail": "Enter a valid expression." }
```

JSON is a text data format. It resembles a JavaScript object, but it is sent as text across the network. HTTP POST means "send data for this action".

Development servers

In development two servers run at the same time. Vite runs React on port 5173; Django runs the API on port 8000. A browser normally blocks a page from one origin from calling another origin. CORS is the server permission that lets the Vite origin call Django during local development.

3. Project map

- Django_calculator/
 - backend/ Django server
 - config/ project-wide routing and settings
 - calculator/ one Django app: calculator domain
 - api/ HTTP endpoint and its URL
 - services/ reusable calculation business logic
 - migrations/ database change history
 - tests/ automated checks
- frontend/ Vite + React browser application

src/App.tsx calculator screen and interaction logic
src/styles/ CSS organized by responsibility
requirements.txt Python packages
run_server.sh convenience script for Django
run_frontend.sh convenience script for Vite
.env private local configuration (do not commit)
.env.example safe template for .env

A Django **project** is the whole server configuration (backend/config). A Django **app** is one focused feature area (backend/calculator). Larger Django projects often have apps such as accounts, orders, and payments alongside their config folder.

4. How one calculation travels through the app

- 1. The user clicks = or presses Enter.
- 2. `input("=")` in `frontend/src/App.tsx` calls `calculate()`.
- 3. `fetch()` makes a POST request, converts { expression } to JSON, and waits with `await` for Django's response.
- 4. Django's top-level `config/urls.py` matches `api/v1/` and includes the calculator app's URLs.
- 5. `calculator/api/urls.py` matches `calculate/` and dispatches to `CalculateView`.
- 6. `CalculateView.post()` verifies that expression is a string and calls `evaluate(expression)`.
- 7. The evaluator tokenizes only allowed numbers and operators, applies normal multiplication/division precedence, and returns a formatted string result.
- 8. Django REST Framework serializes { "result": "14" } as JSON.
- 9. React reads the response, stores the old expression in `previous`, stores the result in `expression`, and React redraws the display.

If any input or network step fails, the frontend shows Error. The backend returns a 400 status for invalid expressions instead of crashing.

5. Backend walkthrough

backend/manage.py

This is Django's command-line entry point. `python manage.py runserver`, `migrate`, `makemigrations`, `test`, and `createsuperuser` all start here. The important line sets `DJANGO_SETTINGS_MODULE` to `config.settings.development`, selecting local-development settings.

backend/config/settings/

`base.py` contains shared configuration. `development.py` imports it and turns on `DEBUG`, local hosts, and `CORS`. `production.py` imports it but keeps `DEBUG` off and adds a few security headers. This separation avoids accidentally deploying developer-friendly settings.

Key choices in `base.py`:

- `BASE_DIR` locates the repository root. `load_dotenv()` reads its `.env`.
- `SECRET_KEY` signs Django security data. The fallback value is acceptable only locally; production must use a long secret from environment variables.
- `INSTALLED_APPS` enables Django built-ins plus `corsheaders`, `DRF` (`rest_framework`), and this project's calculator app.
- `MIDDLEWARE` is an ordered request/response pipeline. `CORS` middleware is

placed early so browser permission headers are added properly.

- DATABASES reads PostgreSQL connection values from environment variables.
- REST_FRAMEWORK restricts this API to JSON in and JSON out.

development.py permits exactly `http://localhost:5173` and `http://127.0.0.1:5173`. If Vite uses another origin, add that precise origin. Do not use a permissive CORS policy in a real deployed application.

Routing: `config/urls.py` and `calculator/api/urls.py`

URL routing maps an incoming path to Python code. The project-level router owns the broad prefixes: `/admin/` is Django's admin; `/api/v1/` belongs to the calculator app. `include()` delegates the remainder, so the app router sees `calculate/`. This keeps each app portable and its URLs together.

`CalculateView.as_view()` converts the view class into a callable Django can route to. DRF then invokes its `post()` method for POST requests.

API view: `calculator/api/views.py`

`APIView` is a DRF base class. `request.data` is parsed JSON; unlike raw Django it is convenient for API bodies. The view implements a deliberately thin controller:

- validate that expression is a string;
- delegate calculation to the service;
- turn success or a known `ExpressionError` into a JSON Response.

`authentication_classes = []` and `permission_classes = []` intentionally make this endpoint public. Add real authentication before attaching calculations to user accounts. The current view does not save a `Calculation` record.

Service: `calculator/services/evaluator.py`

Business logic is outside the view so it can be tested without HTTP and reused by future endpoints, command-line jobs, or other code. The most important security decision is: it does not use Python `eval()`. `eval` would execute arbitrary Python-like input and is unsafe for user data.

The evaluator's algorithm is a small two-stack calculator:

- 1. `strip()` removes edge whitespace and enforces a 200-character maximum.
- 2. `TOKEN_PATTERN` recognizes only decimal numbers and `+`, `-`, `x`, `/`

symbols (the actual code uses multiplication and division glyphs).

- 3. Joining all matched tokens must reproduce the expression exactly. Therefore spaces inside the expression, letters, parentheses, and JavaScript such as `alert(1)` are rejected.

- 4. Number tokens go into values; operator tokens go into pending.
- 5. Before a new operator is added, pending operators of equal or higher

priority are applied. This makes multiplication/division happen before addition/subtraction and makes same-priority operators left-associative.

- 6. `apply()` pops left value, operator, and right value, then pushes the result.
- 7. Division by zero and non-finite results are clear errors. `.12g` formats a

float with up to 12 significant digits and returns a string suitable for the JSON API.

Leading negatives work because `-8/2` is treated as `0-8/2`. Unary negatives in other positions, parentheses, percentage, exponentiation, and scientific functions are not supported yet. That is intentional scope, not a Django limit.

Data model: calculator/models.py

Calculation describes a database table, even though the API currently does not create rows. Django will use the model when history is added.

- id: a UUID primary key; harder to guess than sequential numeric IDs.
- device_id: a client/device identifier, indexed for efficient history lookup.
- expression: what the user typed.
- normalized_expression: a future-friendly canonical version.
- result: saved output.
- angle_mode: deg or rad, ready for future trig functions.
- created_at: set automatically on creation and indexed.

Meta.ordering = ["-created_at"] means newer calculations appear first by default. The composite index (device_id, -created_at) supports the common question "show this device's newest history" efficiently.

Migration: calculator/migrations/0001_initial.py

A migration is a version-controlled recipe for changing the database schema. This initial migration creates the Calculation table and its index. Never casually edit a migration after it has been used by other environments. Instead:

- 1. Change models.py.
- 2. Run python manage.py makemigrations calculator.
- 3. Inspect the new migration.
- 4. Run python manage.py migrate.
- 5. Commit the model and migration together.

Admin and app setup

apps.py identifies this app to Django and defines the default numeric key type for models that do not choose their own key. admin.py registers Calculation, so staff users can inspect it in /admin/ after an admin account is created.

Tests: calculator/tests/test_evaluator.py

SimpleTestCase needs no database, making these tests fast. They check normal precedence, leading negatives, division-by-zero rejection, and rejection of unknown characters. Run them with:

```
cd backend
python manage.py test calculator
```

Useful next tests: decimals, long input, repeated operators, API status codes, and allowed/disallowed CORS origins.

6. Frontend walkthrough

Vite files and packages

package.json lists dependencies and commands. npm run dev starts the development server; npm run build runs TypeScript checks and creates an optimized production bundle in dist/; npm run lint checks code style/rules.

vite.config.ts enables the React plugin. index.html has one important empty element, <div id="root">. React takes control inside that element.

tsconfig.json points to separate browser (tsconfig.app.json) and Vite/Node (tsconfig.node.json) settings. strict: true asks TypeScript to find more mistakes before a browser sees them. noEmit: true means TypeScript checks types while Vite handles the actual

bundling.

src/main.tsx

This is the frontend entry point. `createRoot(...).render(...)` starts React in the root div. `StrictMode` adds development-only checks and can intentionally re-run certain work to expose unsafe side effects. `BrowserRouter` prepares client-side routing; this small app has no routes yet, so it is future-ready. Finally, the shared CSS entry file is imported once.

React state in src/App.tsx

A React component is a function that returns UI. React calls it again after a state update, then efficiently updates only the DOM differences.

`useState` creates values that survive renders:

- `expression`: the currently visible number/expression, initially "0".
- `previous`: the last completed expression for the smaller display line.
- `justCalculated`: tells digit entry to begin a new expression after `=`.
- `isCalculating`: prevents duplicate input while the API request is active.

`setExpression` does not change the screen instantly in-place. It asks React to schedule a new render. The functional form, such as `setExpression((value) => ...)`, is used when the new value depends on the previous value; that avoids stale-state problems.

Operator is a TypeScript union: only four exact strings are valid operators. The operators array is used to recognize them at runtime.

Button input behavior

`input(key)` centralizes all calculator behavior, so mouse buttons and the keyboard act consistently. It handles clear, backspace, equals, sign toggle, operator replacement, one decimal point per current number, and regular digit entry. The buttons are data in an array and `.map()` turns each into a React `<button>`, avoiding repeated markup.

The class name chooses a visual type such as `key--operator` or `key--equals`. The button's `key` prop helps React identify each item in the mapped list; it does not become an HTML attribute for the user.

Calling Django with fetch

`calculate` is marked `async`, so it can use `await`. It:

1. sets the loading state;
2. uses `fetch` with `POST`, a `JSON` content-type header, and a `JSON` body;
3. parses the `JSON` response;
4. rejects either non-2xx `HTTP` responses or missing results;
5. updates display state on success, otherwise shows `Error`;
6. always clears loading in finally.

The endpoint is configurable through `VITE_CALCULATOR_API_URL`. Vite exposes environment variables prefixed `VITE_` to client-side code. This matters: never put a secret in a `VITE_` variable because the browser can read it.

There is a configuration mismatch worth fixing: `.env.example` defines `VITE_API_BASE_URL`, while `App.tsx` reads `VITE_CALCULATOR_API_URL`. The latter works through its `localhost` fallback, but an environment file following the example will not override the URL. Choose one name and use it in both places.

Keyboard support and effects

`useEffect` is for work outside React rendering. Here it registers a browser `keydown` listener after React renders and returns a cleanup function that removes it. That cleanup prevents

duplicate listeners after later renders or when the component unmounts. The dependency `[isCalculating]` refreshes the listener when the loading state changes.

The mapping translates standard keyboard `*` and `/` into the screen's `x` and division symbols, maps Enter to equals, Escape to clear, and Backspace to the backspace control. `preventDefault()` avoids a browser default behavior such as navigating back on Backspace.

CSS organization

CSS is split by responsibility:

- `tokens.css`: global design tokens and automatic light/dark color variables.
- `reset.css`: predictable browser defaults, including box-sizing.
- `layout.css`: root and application minimum heights.
- `calculator.css`: this calculator component's visual rules.
- `index.css`: imports the first three shared layers.

The calculator layout uses CSS Grid. `grid-template-columns: repeat(4, 1fr)` creates four equal columns. The equals key spans two columns. `clamp()` keeps the display type responsive, `min()` keeps the card from becoming too wide, and the media query adjusts spacing on narrow screens. `:focus-visible` is an important keyboard-accessibility detail: a focused key has a visible outline.

7. Running the project locally

Prerequisites are Python 3.12+, Node.js 20+, and PostgreSQL 16+ according to the project README. In the project root:

```
cp .env.example .env
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cd backend
python manage.py migrate
python manage.py runserver
```

In another terminal:

```
cd Django_calculator/frontend
npm install
npm run dev
```

Open the Vite URL (normally `http://localhost:5173`). You need PostgreSQL running with credentials matching `.env` before migrate or server database checks can succeed. Keep `.env` private; it contains secrets and passwords.

The helper scripts do the same basic work. `run_server.sh` activates `.venv`, migrates, then starts Django. `run_frontend.sh` installs packages then starts Vite. For faster repeated frontend starts, use `npm run dev` directly after the first install. The scripts currently contain absolute `/root/...` paths, so they need updating if the project is moved or shared with another developer.

8. Debugging checklist

If the screen shows Error, use the browser DevTools Network tab. Confirm the request URL, request JSON, HTTP status, and response body. A browser-console CORS message usually means the Vite origin is absent from `CORS_ALLOWED_ORIGINS`.

If Django will not start, first activate the virtual environment, then confirm `pip install -r requirements.txt` completed. Configuration errors often come from a missing `.env` or unavailable PostgreSQL database. Use `python manage.py check` for Django's built-in

configuration checks.

If React does not refresh, check the terminal running Vite and the browser console. Run `npm run build` for TypeScript errors and `npm run lint` for lint errors. The exact error message is usually more useful than guessing.

9. A repeatable blueprint for future projects

Use the same separation of responsibilities:

React component -> API client/function -> Django view -> service -> model/database

For a new feature, begin with a small user story, then define the API contract before UI details. Keep views thin; put real rules in named service functions; validate every external input on the server; add tests for the service and API; and make environment-dependent values configurable rather than hard-coded.

For example, to add calculation history:

- 1. Decide the API shape, e.g. POST returns a saved id and GET returns recent calculations for a device or authenticated user.
- 2. Validate a device id (or introduce Django authentication).
- 3. In the view, call `evaluate`, then create `Calculation.objects.create(...)`.
- 4. Add a list endpoint, queryset filtering, pagination, and serializers.
- 5. Add frontend state for history, a fetch on load, loading/error UI, and a history component.
- 6. Add tests before relying on the feature.

For more calculator operations, do not expand the API view into a long parser. Extend or replace the evaluator with a clearly tested tokenizer/parser. Decide which grammar you support, limit input size/complexity, and keep arbitrary-code execution impossible.

10. Learning path and useful commands

Learn in this order: HTML/CSS basics; JavaScript functions, arrays, promises, and `async/await`; React components/state/effects; HTTP and JSON; Python functions/exceptions; Django URLs/views/models/migrations; then tests, authentication, deployment, and security.

```
# Backend
cd backend
python manage.py test calculator
python manage.py check
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser

# Frontend
cd frontend
npm run dev
npm run build
npm run lint
```

11. Key takeaways

The frontend should manage presentation and user interaction; the backend must be the source of truth for rules and validation. Data crosses the boundary as a small documented API contract. Keep code focused: route URLs, views coordinate, services hold business logic, models describe stored data, migrations record schema changes, and tests protect behavior. This calculator is a concise but solid template for many future full-stack projects.